Inteligență Artificială: Tema 2

Utilizarea modelelor neurale pentru clasificarea imaginilor

Asignată: 07 Mai 2026	**Deadline: 24 Mai 2026**

1 Descriere generală

În practica de zi cu zi a unui inginer în IA, metodele clasice de învățare automată sunt frecvent completate sau înlocuite cu modele bazate pe **rețele neurale** și instrumentele moderne de antrenare a acestora. Utilizarea rețelelor neurale presupune familiarizarea cu un set de aspecte practice care trebuie stăpânite:

- **Construirea unui pipeline complet de antrenare** (dataset, dataloader, optimizator, funcție de loss, bucla de train/eval) și jurnalizarea evoluției pe parcursul epocilor (curbele de loss și de performanță)
- **Monitorizarea și compararea sistematică a mai multor experimente** — diferite arhitecturi (MLP vs CNN), diferite configurații de hiperparametri — folosind unelte dedicate precum TensorBoard
- **Augmentarea datelor** pentru a compensa dimensiunea redusă a seturilor de antrenare și pentru a îmbunătăți generalizarea
- **Tratarea seturilor de date dezechilibrate** prin tehnici de over-/under-sampling la nivel de batch și prin ajustarea funcției de loss (weighted cross entropy, focal loss)
- **Reutilizarea modelelor pre-antrenate prin fine-tuning (transfer learning)**, inclusiv adaptarea formei intrării la cea așteptată de model (rezoluție, număr de canale — în special situația imaginilor grayscale folosite cu modele pre-antrenate pe ImageNet) și folosirea unor strategii moderne de optimizare precum learning rate warmup
- **Aplicarea unor paradigme de antrenare mai avansate**, precum curriculum learning, prin care exemplele sunt prezentate modelului într-o ordine de dificultate crescătoare

Scopul acestei teme constă în familiarizarea cu conceptele de bază ale antrenării rețelelor neurale în PyTorch, abordarea cap-coadă a unei probleme de viziune artificială (computer vision), jurnalizarea riguroasă a seturilor de experimente, precum și analiza și susținerea observațiilor dobândite pe parcursul antrenării și fine-tuning-ului modelelor. În **Anexă** găsiți un tutorial bazat pe fragmente de cod care acoperă aspectele relevante enumerate mai sus și care vă va servi ca punct de plecare pentru rezolvarea sarcinilor.

2 Descrierea Seturilor de Date

Sarcinile de lucru solicită utilizarea bibliotecii **PyTorch** pentru implementarea, antrenarea și evaluarea modelelor de tip MLP și CNN pe două probleme de clasificare a imaginilor: **recunoașterea emoțiilor din expresii faciale** și **clasificarea aluniţelor de piele – eng. skin moles**. Cele două probleme au fost alese pentru a vă expune la dificultăți frecvent întâlnite — variabilitate, seturi de date mici (între 10k – 20k imagini) și dezechilibrate, în care erorile pe clasele minoritare pot avea cost ridicat.

2.1 Setul de date “Vai de pielea mea”

Acest set de date reprezintă o colecție vastă de imagini dermatoscopice ale leziunilor cutanate pigmentate, colectate pe parcursul unei perioade lungi de timp. Setul a fost creat pentru a antrena și testa algoritmi de inteligență artificială capabili să identifice rapid și automat diverse tipuri de cancer de piele. Conține 10015 imagini color de 130x100 pixeli. Setul de date este dezechilibrat, unele tipuri de leziuni sunt mult mai frecvente decât altele.
Structura datelor:

1. Imaginile medicale: Fotografii macro realizate prin dermatoscopie.
2. Eticheta (Clasa țintă): Coloana “diagnostic”, care indică diagnosticul confirmat prin histopatologie sau expertiză medicală.

Descrierea claselor

Clasa	Descriere
akiec	Bowel disease
bcc	Carcinom bazocelular
bkl	Leziuni benigne de tip keratoză
df	Dermatofibrom
mel	Melanom
nv	Nevi melanocitari (alunițe obișnuite)
vasc	Leziuni vasculare

Train are 6009, test are 2003, iar kaggle_test are 2003 de sample-uri.
Valori pentru pentru normalizarea și standardizarea datelor (RGB):

- mean = [0.7646, 0.5464, 0.5712]
- std = [0.1410, 0.1532, 0.1706]

2.2 Setul de date “You’re on candid camera”

Acesta este un set de date pentru recunoașterea expresiilor faciale, conținând 15.339 de imagini color de 100×100 pixeli, colectate în condiții reale și necontrolate. Imaginile sunt organizate în 7 clase de emoții de bază, prezentând un dezechilibru semnificativ (Happiness: ~38.9%, Fear: ~2.3%) și variabilitate ridicată în iluminare, ocluzie și diversitate etnică.

Mapare clase: 1=Surprise, 2=Fear, 3=Disgust, 4=Happiness, 5=Sadness, 6=Anger, 7=Neutral
Statistici per clasă :

Clasă	Emoție	Train	Local Test	Remote Test
1	Surprise	1097 (10.5%)	193 (10.5%)	329 (10.7%)
2	Fear	239 (2.3%)	42 (2.3%)	74 (2.4%)
3	Disgust	610 (5.8%)	107 (5.8%)	160 (5.2%)
4	Happiness	4057 (38.9%)	715 (38.9%)	1185 (38.6%)
5	Sadness	1685 (16.1%)	297 (16.2%)	478 (15.6%)
6	Anger	600 (5.8%)	105 (5.7%)	162 (5.3%)
7	Neutral	2146 (20.6%)	378 (20.6%)	680 (22.2%)

Train are 10.434, Local Test are 1.837, iar Remote Test are 3.068 de sample-uri.
Arhiva cuprinde:

- DATASET/train/ și DATASET/test/ (imaginile originale; aliniate facial)
- splits/train.csv (set de antrenare cu 2 coloane, adică (cale, etichetă))
- splits/local_test.csv (set pentru test local cu 2 coloane, adică (cale, etichetă))
- splits/remote_test.csv (test remote pe Kaggle, fără etichete (are doar id-uri))
- splits/norm_stats.json, medie și deviație standard per canal RGB, calculate exclusiv pe train

3 Cerințe

3.1 Vizualizarea augmentării seturilor de date [1p]

Implementați augmentări **potrivite** pentru imaginile din fiecare dintre cele două seturi de date. **Justificați în raport** utilitatea oricărui tip de augmentare ales și exemplificați în mod vizual rezultatul aplicării acesteia pe câte cel puțin un exemplu din fiecare clasă al fiecărui set de date.

Pentru documentarea asupra și implementarea unui set larg de augmentări pentru imagini, puteți folosi biblioteca Albumentations.

3.2 Arhitectură MLP [1.5p]

Implementați, antrenați și evaluați o arhitectură de tip MLP pe fiecare din cele două seturi de date. Documentați și motivați în raport arhitectura folosită (număr straturi, dimensiunea fiecărui strat, funcții non-liniare utilizate, straturi de normalizare - e.g. BatchNormalization, regularizare prin dropout).

3.3 Arhitectură CNN [1.5p]

Implementați, antrenați și evaluați o arhitectură de tip CNN **comună** celor două seturi de date.
Se impun următoarele restricții:

- cel mult 4 straturi convoluționale
- cel mult 2 straturi liniare complet conectate

Documentați și motivați în raport arhitectura folosită (număr straturi, parametrizarea straturilor de convoluție, funcții non-liniare utilizate, straturi de normalizare - e.g. BatchNormalization, straturi de pooling – incluzand global average pooling, regularizare prin dropout).

Luați în considerare următoarele aspecte în dezvoltarea arhitecturii:

- **Consideraţii despre intrare:** trebuie să stabiliți **forma intrării**, pentru că ea dictează direct dimensionarea straturilor. Două aspecte sunt importante:
  - **Standardizarea dimensiunii.** Imaginile dintr-un dataset rareori au exact aceeaşi rezoluție, dar un CNN clasic (cu straturi Linear la final) trebuie să primească intrări de aceeaşi dimensiune în cadrul unui batch. Soluții uzuale:
    - transforms.Resize((H, W)) — alegem o dimensiune fixă, comună pentru întregul set. Pentru cele două probleme alegem o dimensiune *apropiată de cea originală* a imaginilor (e.g. 128×128 sau 96×96).
    - Alternativă: nn.AdaptiveAvgPool2d((1,1)) la finalul părții convoluționale — astfel rețeaua devine *agnostică la rezoluția de intrare* (atât timp cât este suficient de mare ca feature map-urile să nu colapseze înainte). **Aceasta este soluția modernă, recomandată**.

- **Particularizarea primului strat.** Cele două seturi de date au caracteristici diferite, iar primul strat convoluțional este cel care „vede direct" pixelii — deci este punctul natural în care le tratăm diferit:
  - Expresiile faciale sunt color (RGB, 3 canale), iar caracteristicile relevante (margini ale ochilor, gurii, sprâncenelor) sunt de scară mică → un kernel 3×3 cu stride 1 este potrivit.
  - **Aluniţe canceroase**: sunt dermoscopice color (RGB, 3 canale), iar trăsăturile relevante (asimetrie, neregularitatea conturului, structuri pigmentare) au scară puțin mai mare → putem folosi un prim kernel 5×5 sau două straturi 3×3 consecutive pentru a crește receptive field-ul rapid.

3.4 Finetuning [2p]

Rezolvați cele două probleme de clasificare folosind procedura de **ajustare** a unei **rețele preantrenate** (Vedeți Secțiunea 6 din Anexă și tutorialul de Transfer Learning in Computer Vision al PyTorch).
Pentru fine tuning, vă este recomandată folosirea uneia dintre următoarele arhitecturi existente:

- ResNet-18 – detalii arhitecturale aici
- MobileNet-v2 – detalii arhitecturale aici

3.5 Rezultate antrenării evaluate prin ablații [4p]

Folosiți **metodologia** raportării rezultatelor prin teste de ablație **explicată în Tema 1.**
Se cer următoarele analize:

- Arhitectura MLP **cu** / **fără** straturi de dropout **[0.5p]**
- Arhitectura CNN **cu / fără** straturi de BatchNorm (**vedeți Secțiunea 5** din Anexă) **[0.5p]**
- Antrenarea arhitecturii CNN sub diferite configurații de augmentare a datelor (**vedeți Secțiunea 3** din Anexă) [**1p**]
  - fără augmentări
  - cu augmentări geometrice (e.g. flip-uri, scalări + crop-uri, rotiri)
  - cu augmentări în spațiul de culoare (e.g. blurare, color jitter)
  - cu toate augmentările
- Antrenarea **cu / fără** tehnici de rezolvare a dezichilbrului de clase (**vedeți Secțiunea 4** din Anexă) [1p]
- Finetuning al unei rețele preantrenate prin **înghețarea backbone-ului, cu / fără** learning rate warmup (**vedeți Sectiunea 6** din Anexă) [**1p**]

4 Trimiterea temei și metodă punctare

O trimitere completă a temei constă în:

- O **arhivă zip** sau un **Jupyter Notebook** care conține implementarea analizelor cerute. Arhiva sau notebook-ul respectă convenția de numire *Tema2-IA-<nume>-<prenume>-cod.zip(ipynb)*
- Un **fișier PDF** care prezintă **raportul de rezultate și interpretare** a analizelor cerute. Raportul respectă convenția de numire *Tema2-IA-<nume>-<prenume>-raport.pdf*

Raportul trebuie să conțină obligatoriu:

- **Descrierea** arhitecturii MLP, **documentarea** metodei de antrenare (numar epoci, optimizator, lr scheduling), **graficele curbelor de train / test** și **rezultatele** ablației de folosire a straturilor de dropout
- **Descrierea** arhitecturii CNN, **documentarea** metodelor de antrenare, **graficele curbelor de train / test** si **rezultatele** ablațiilor după:
  - Folosirea straturilor de normalizare
  - Tipurile de augmentare
- **Graficele curbelor de train / test și rezultatul** ablației dupa tehnici de rezolvare a dezechilibrului de clase
- **Documentarea** procedurii de finetuning folosită, **graficele curbelor de train / test** și **rezultatele** ablației cu / fără learning rate warm-up

! Atenție: **Rezultat se referă la:**

- grafice cu curbele de train și test (ca evaluare a procedurii de antrenare)
- documentare a cazurilor de overfit / underfit
- raportarea unei **matrice de confuzie** si a **metricilor *accuracy, F1-macro, F1-micro*** pe fiecare din cele două seturi de date

! Atenție: **orice rezultat trebuie comentat în textul raportului (cel puțin 1-2 fraze)**

! Atenție: **orice analiză de ablație trebuie susținută de un grafic / un tabel pe seama căruia să se poată face o analiză comparativă (i.e. comparația se face “văzând direct pe graf / în tabel”).**

! Atenție: **punctajul pe cerințele de la punctul 3.5 sunt condiționate de obținerea pe platforma Kaggle a unui scor F1-macro ce depășesc pragurile aferente fiecărui subpunct (MLP, CNN, finetuning) pe care le vom anunța pe Moodle.**

4.1 BONUS

Se vor acorda 2 puncte BONUS pentru analizele care evaluează impactul unor tehnici de **curriculum learning** pe unul dintre cele două seturi de date.
Pentru ideea generală și proceduri de curricum learning vedeți Secțiunea 7 din Anexă.

Anexă: Tutorial PyTorch pentru antrenarea MLP-urilor și CNN-urilor

Această anexă este un tutorial practic, bazat pe fragmente de cod, care acoperă aspectele esențiale necesare pentru rezolvarea temei (recunoașterea fețelor și clasificarea mole-urilor cutanate pe imagini de rezoluție mică). Fragmentele de cod **nu** sunt gândite să fie rulate cap-coadă ca un singur program — în schimb, fiecare fragment ilustrează un concept specific, urmând să le combinați într-un pipeline propriu de antrenare.

Toate comentariile din cod sunt scrise în limba română.

---

1. Bucla de antrenare „standard”

Pipeline-ul „standard” de antrenare în PyTorch urmează mereu același schelet: **Dataset → DataLoader → Model → Loss → Optimizer → Bucla de train/eval → Logging.** Odată ce ați înțeles acest schelet, trecerea între un MLP și un CNN, sau între recunoașterea fețelor și clasificarea mole-urilor, devine o chestiune de a schimba câteva componente.

1.1 Dataset și DataLoader

Clasa `Dataset` se ocupă de încărcarea **unui singur** exemplu pe rând. `DataLoader`-ul o învelește și se ocupă de batching, shuffle și încărcare paralelă.

```
import torch
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import os

# Definim un Dataset custom; pentru tema voastra imaginile pot fi
# fete (face recognition) sau mole-uri (skin lesion classification).
class ImageFolderDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        # Memoram lista de (cale_imagine, eticheta) pentru a o folosi in __getitem__
        self.samples = []
        self.transform = transform
        # Fiecare subfolder reprezinta o clasa (ex: clasa "0", clasa "1", ...)
        classes = sorted(os.listdir(root_dir))
        self.class_to_idx = {c: i for i, c in enumerate(classes)}
        for c in classes:
            cls_dir = os.path.join(root_dir, c)
            for fname in os.listdir(cls_dir):
                self.samples.append((os.path.join(cls_dir, fname), self.class_to_idx[c]))

    def __len__(self):
        # PyTorch are nevoie sa stie cate exemple sunt in dataset
```

```
        return len(self.samples)

    def __getitem__(self, idx):
        # Incarcam imaginea si eticheta pentru indexul cerut
        path, label = self.samples[idx]
        img = Image.open(path).convert("RGB")  # ne asiguram ca avem 3 canale
        if self.transform is not None:
            # Aplicam transformarile (resize, normalizare, augmentari, etc.)
            img = self.transform(img)
        return img, label
```

```
from torchvision import transforms

# Transformari minimale: resize la o dimensiune fixa, conversie in tensor si normalizare.
# Pe imaginile de tema (rezolutie mica) folositi o dimensiune apropiata de cea originala.
basic_transform = transforms.Compose([
    transforms.Resize((64, 64)),                        # uniformizam dimensiunea
    transforms.ToTensor(),                              # PIL -> Tensor cu valori in [0, 1]
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3),    # normalizare simpla in [-1, 1]
])

train_set = ImageFolderDataset("data/train", transform=basic_transform)
val_set   = ImageFolderDataset("data/val",   transform=basic_transform)

# DataLoader-ul se ocupa de batching, shuffle si paralelizare
train_loader = DataLoader(train_set, batch_size=64, shuffle=True,  num_workers=2)
val_loader   = DataLoader(val_set,   batch_size=64, shuffle=False, num_workers=2)
```

1.2 Model, funcție de loss, optimizator

```
import torch.nn as nn
import torch.optim as optim

# Exemplu de MLP simplu pentru imagini 64x64 RGB (3*64*64 = 12288 features de intrare)
class SimpleMLP(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),                # transformam imaginea in vector
```

```
            nn.Linear(3*64*64, 512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),              # dropout pentru regularizare
            nn.Linear(512, num_classes), # iesirea = scoruri (logits) per clasa
        )

    def forward(self, x):
        return self.net(x)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = SimpleMLP(num_classes=2).to(device)

# CrossEntropyLoss combina LogSoftmax + NLLLoss; primeste logits, NU probabilitati
criterion = nn.CrossEntropyLoss()

# Adam este o alegere sigura pentru inceput; SGD+momentum poate fi mai bun cu tuning
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
```

1.3 Bucla de antrenare și evaluare

Bucla de antrenare iterează pe epoci; în fiecare epocă parcurgem setul de date complet o singură dată. După fiecare epocă, evaluăm pe setul de validare și logăm metricile pentru a putea reprezenta grafic ulterior **curba de loss** și **curba de acuratețe**.

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    # Setam modelul pe modul "train" -> activeaza dropout, batchnorm in mod statistic
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        # 1) Resetam gradientii (altfel se acumuleaza de la batch-ul precedent)
        optimizer.zero_grad()
        # 2) Forward pass
        logits = model(images)
        # 3) Calculam loss-ul intre predictii si etichetele reale
        loss = criterion(logits, labels)
        # 4) Backward pass -> calculeaza gradientii
        loss.backward()
        # 5) Pas de optimizare -> actualizeaza parametrii
        optimizer.step()
```

```
        # Statistici pentru logging
        running_loss += loss.item() * images.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    # Returnam loss mediu si acuratete pe epoca
    return running_loss / total, correct / total
```

```
@torch.no_grad()  # dezactivam autograd -> mai rapid si consuma mai putina memorie
def evaluate(model, loader, criterion, device):
    # Modul "eval" -> dezactiveaza dropout si foloseste running stats la batchnorm
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        loss = criterion(logits, labels)

        running_loss += loss.item() * images.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    return running_loss / total, correct / total
```

```
# Bucla principala de antrenare cu logging in liste (pentru a face plot-uri ulterior)
num_epochs = 20
history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}

for epoch in range(num_epochs):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, optimizer, device)
    va_loss, va_acc = evaluate(model, val_loader, criterion, device)

    history["train_loss"].append(tr_loss); history["train_acc"].append(tr_acc)
    history["val_loss"].append(va_loss);   history["val_acc"].append(va_acc)

    print(f"Epoca {epoch+1:02d} | "
          f"train loss={tr_loss:.4f} acc={tr_acc:.3f} | "
```

```
          f"val loss={va_loss:.4f} acc={va_acc:.3f}")

# Salvam modelul cel mai bun (cea mai mica val_loss) - exemplu simplificat:
torch.save(model.state_dict(), "best_model.pt")
```

```
import matplotlib.pyplot as plt

# Plot-uri pentru curba de loss si curba de acuratete
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(history["train_loss"], label="train")
axes[0].plot(history["val_loss"],   label="val")
axes[0].set_title("Loss"); axes[0].set_xlabel("epoca"); axes[0].legend()

axes[1].plot(history["train_acc"], label="train")
axes[1].plot(history["val_acc"],   label="val")
axes[1].set_title("Accuracy"); axes[1].set_xlabel("epoca"); axes[1].legend()
plt.tight_layout(); plt.show()
```

Resursă utilă: Tutorialul oficial PyTorch — Quickstart

---

2. Monitorizarea antrenării cu TensorBoard

Atunci când începeți să comparați mai multe arhitecturi (MLP vs CNN) sau mai multe configurații de hiperparametri (rata de învățare, dimensiunea batch-ului, dropout), afișarea numerelor în terminal devine greoaie. TensorBoard salvează totul pe disc, iar voi puteți răsfoi curbele interactiv în browser.

Ideea cheie este să folosiți un **`log_dir` separat pentru fiecare experiment** — fiecare rulare va apărea ca o curbă diferită în TensorBoard.

2.1 Crearea unui SummaryWriter

```
from torch.utils.tensorboard import SummaryWriter
from datetime import datetime

# Numele rulei trebuie sa fie descriptiv; recomandare: include arhitectura + hiperparametrii cheie.
# In acest fel, cand deschideti TensorBoard veti putea distinge usor experimentele.
run_name = f"mlp_lr1e-3_bs64_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
writer = SummaryWriter(log_dir=f"runs/{run_name}")
```

2.2 Logarea valorilor scalare în timpul antrenării

```
# Varianta imbunatatita a buclei de antrenare care logheaza in TensorBoard
for epoch in range(num_epochs):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, optimizer, device)
    va_loss, va_acc = evaluate(model, val_loader, criterion, device)

    # Scalars: o singura valoare per epoca, grupata logic prin "tag-ul" cu /
    # Astfel "Loss/train" si "Loss/val" vor aparea pe acelasi grafic in TensorBoard.
    writer.add_scalar("Loss/train", tr_loss, epoch)
    writer.add_scalar("Loss/val",   va_loss, epoch)
    writer.add_scalar("Acc/train",  tr_acc,  epoch)
    writer.add_scalar("Acc/val",    va_acc,  epoch)

    # Logam si learning rate-ul (util cand folositi scheduler)
    writer.add_scalar("LR", optimizer.param_groups[0]["lr"], epoch)

writer.close()  # important: inchidem writerul la final ca sa se scrie tot pe disc
```

2.3 Logarea de imagini, histograme și hiperparametri

```
# Imagini: utile pentru a vedea cum arata batch-urile DUPA augmentari.
images, labels = next(iter(train_loader))
import torchvision
grid = torchvision.utils.make_grid(images[:16], nrow=4, normalize=True)
writer.add_image("Train/sample_batch", grid, global_step=0)

# Histograme: utile pentru a vedea distributia ponderilor sau a gradientilor.
for name, param in model.named_parameters():
    writer.add_histogram(f"weights/{name}", param, global_step=epoch)
    if param.grad is not None:
        writer.add_histogram(f"grads/{name}", param.grad, global_step=epoch)

# Hiperparametri: tab-ul HPARAMS permite compararea mai multor configuratii.
writer.add_hparams(
    {"lr": 1e-3, "batch_size": 64, "model": "MLP"},
    {"hparam/val_acc": va_acc, "hparam/val_loss": va_loss},
)
```

2.4 Pornirea TensorBoard

În terminal, din rădăcina proiectului:

```
# Pornim TensorBoard pe directorul "runs/" - va citi automat toate experimentele de acolo
tensorboard --logdir runs
# Apoi deschideti in browser: http://localhost:6006
```

Resursă utilă: PyTorch — Vizualizarea modelelor, datelor și antrenării cu TensorBoard

---

3. Augmentarea datelor cu `torchvision.transforms`

Augmentarea datelor extinde artificial setul de antrenare aplicând transformări aleatoare „on the fly” (la fiecare epocă, fiecare imagine apare puțin diferit). Acest lucru este **esențial** pe seturi mici de date, cum sunt cele din temă — atât fețele, cât și mole-urile beneficiază enorm de pe urma augmentării. Regula de bază este: **augmentările se aplică doar pe setul de antrenare**, niciodată pe validare/test.

3.1 Construirea unui pipeline pentru antrenare și unul pentru validare

```
from torchvision import transforms

# Pe setul de antrenare: augmentari aleatoare la fiecare epoca
train_transform = transforms.Compose([
    transforms.Resize((72, 72)),              # putin mai mare decat dimensiunea finala
    transforms.RandomCrop(64),                # crop aleator -> introducem variabilitate
    transforms.RandomHorizontalFlip(p=0.5),   # OK pentru mole-uri si pentru fete
    transforms.ColorJitter(brightness=0.2,
                           contrast=0.2,
                           saturation=0.2),   # variatii de culoare/lumina
    transforms.RandomRotation(degrees=15),    # rotatii mici
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),  # statistici ImageNet
])

# Pe setul de validare/test: NICIO augmentare aleatoare!
# Folosim doar resize + ToTensor + Normalize, identic cu ceea ce facem la inferenta.
val_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
```

```
                        std=[0.229, 0.224, 0.225]),
])
```

3.2 Observații specifice fiecărei probleme

```
# Atentie la augmentarile care schimba semantica problemei!
# - Pentru recunoasterea fetelor: RandomVerticalFlip e o idee PROASTA (fetele nu apar cu capul in jos).
# - Pentru clasificarea mole-urilor: rotatiile la 90/180/270 grade SUNT OK
#   (un mole nu are o orientare "naturala"), dar ColorJitter prea agresiv poate sterge
#   exact culorile care conteaza pentru diagnostic (melanom vs nev benign).

# Exemplu mai potrivit pentru imagini medicale (mole-uri):
mole_train_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),                # OK: nu exista "sus/jos" pentru mole
    transforms.RandomRotation(degrees=180),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),  # subtil!
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
```

3.3 Vizualizarea augmentărilor înainte de antrenare

```
# Inainte de antrenare, vizualizati cateva exemple augmentate ca sa va asigurati
# ca transformarile sunt rezonabile (nu prea agresive, nu schimba clasa).
import matplotlib.pyplot as plt

img, label = train_set[0]  # un sample din dataset (cu train_transform aplicat)
fig, axes = plt.subplots(1, 6, figsize=(12, 3))
for ax in axes:
    img_aug, _ = train_set[0]            # se reaplica augmentarile aleator de fiecare data
    ax.imshow(img_aug.permute(1, 2, 0) * 0.5 + 0.5)
    ax.axis("off")
plt.show()
```

Resursă utilă: Documentația și ilustrațiile pentru `torchvision.transforms`

---

4. Tratarea dezechilibrului între clase: sampling și ajustarea funcției de loss

În seturile de date medicale (mole-uri), de regulă aveți mult mai multe cazuri benigne decât maligne. Dacă ignorați acest lucru, rețeaua va învăța să prezică mereu „benign” și totuși va obține o acuratețe mare — ceea ce este inutil. Există două familii mari de soluții: **schimbăm ce vede rețeaua** (sampling) și **schimbăm ce optimizează rețeaua** (funcții de loss ponderate).

4.1 WeightedRandomSampler — over/under-sampling la nivel de batch

```
import numpy as np
from torch.utils.data import WeightedRandomSampler

# Numaram cate exemple are fiecare clasa
labels = np.array([label for _, label in train_set.samples])
class_counts = np.bincount(labels)              # ex: [900, 100] -> dezechilibru 9:1
print("Distributia claselor:", class_counts)

# Calculam o pondere INVERS proportionala cu frecventa clasei.
# Clasa rara va primi pondere mare -> va fi extrasa mai des.
class_weights = 1.0 / class_counts              # ex: [1/900, 1/100]
sample_weights = class_weights[labels]          # ponderea fiecarui exemplu in parte

# replacement=True -> exemplele din clasa rara se vor repeta in cadrul unei epoci (oversampling).
# replacement=False ar face mai degraba undersampling al clasei majoritare.
sampler = WeightedRandomSampler(
    weights=torch.from_numpy(sample_weights).double(),
    num_samples=len(sample_weights),
    replacement=True,
)

# IMPORTANT: cand folosim sampler, NU mai punem shuffle=True (sampler-ul decide ordinea).
balanced_loader = DataLoader(train_set, batch_size=64, sampler=sampler, num_workers=2)
```

4.2 Cross Entropy ponderat

```
# Alternativa (sau complementara): pastram dataloader-ul normal, dar penalizam mai tare
# greselile pe clasa rara prin ponderi in functia de loss.
counts = torch.tensor(class_counts, dtype=torch.float)
# Formula uzuala: w_c = N_total / (num_classes * N_c)
weights = counts.sum() / (len(counts) * counts)
weights = weights.to(device)
```

```
# CrossEntropyLoss accepta direct un vector de ponderi per clasa
criterion = nn.CrossEntropyLoss(weight=weights)
```

4.3 Focal Loss

Focal Loss reduce ponderea exemplelor clasificate corect cu încredere mare, astfel încât modelul să se concentreze pe exemplele dificile, adesea din clasa minoritară. Este foarte populară în imagistica medicală.

```
import torch.nn.functional as F

class FocalLoss(nn.Module):
    """
    Focal Loss = - alpha * (1 - p_t)^gamma * log(p_t)
    - gamma: cu cat e mai mare, cu atat se concentreaza mai mult pe exemplele grele
    - alpha: pondere per clasa (similara cu weighted CE), optionala
    """
    def __init__(self, alpha=None, gamma=2.0, reduction="mean"):
        super().__init__()
        self.alpha = alpha       # tensor de marime [num_classes] sau None
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, logits, targets):
        # CE per-exemplu, fara reducere -> ne trebuie p_t pentru fiecare sample
        ce = F.cross_entropy(logits, targets, weight=self.alpha, reduction="none")
        p_t = torch.exp(-ce)                    # p_t = probabilitatea clasei corecte
        loss = (1 - p_t) ** self.gamma * ce     # factorul "focal"
        if self.reduction == "mean":
            return loss.mean()
        elif self.reduction == "sum":
            return loss.sum()
        return loss

criterion = FocalLoss(alpha=weights, gamma=2.0)
```

4.4 Sfaturi practice

```
# Recomandari practice:
# - Incepeti cu weighted CE (cea mai simpla schimbare).
```

```
# - Daca tot aveti recall slab pe clasa rara, treceti la WeightedRandomSampler.
# - Combinarea sampler + weighted loss este posibila, dar poate dezechilibra in cealalta directie.
# - Monitorizati metrici per-clasa (precision/recall/F1), NU doar accuracy globala -
#   pe un set 90/10, un model care prezice mereu majoritar are 90% accuracy dar este inutil.
```

Resursă utilă: Dezechilibrul claselor în PyTorch — ghid practic (Geeks for Geeks)

---

5. Batch Normalization în CNN-uri

Batch Normalization (BatchNorm) este una dintre cele mai utile „invenții" din istoria recentă a rețelelor neuronale: stabilizează antrenarea, permite folosirea unor learning rate-uri mai mari, accelerează convergența și are un efect ușor de regularizare. Pe scurt, după fiecare strat convoluțional, BatchNorm normalizează ieșirile (le aduce la medie 0 și varianță 1 pe fiecare canal, calculat peste batch-ul curent) și apoi aplică o transformare afină învățabilă (`gamma`, `beta`) — astfel rețeaua poate „anula" normalizarea dacă are nevoie.

Pentru imaginile din temă (rezoluție mică, seturi de date relativ mici), BatchNorm face de obicei diferența între un CNN care învață greu și unul care converge frumos.

5.1 Adăugarea BatchNorm într-un CNN simplu

```
import torch.nn as nn

# Pattern-ul standard intr-un CNN modern: Conv -> BatchNorm -> Activare -> (eventual Pool)
# IMPORTANT: bias=False la Conv2d cand este urmat de BatchNorm,
# pentru ca BN are deja propriul "beta" care joaca rolul de bias.
class SimpleCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(
            # Block 1
            nn.Conv2d(3, 32, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(32),                 # cate un set de (gamma, beta) per canal
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                    # 64x64 -> 32x32

            # Block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),                    # 32x32 -> 16x16
```

```
            # Block 3
            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d(1),            # global average pooling -> (B, 128, 1, 1)
        )
        self.classifier = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)                        # (B, 128, 1, 1) -> (B, 128)
        return self.classifier(x)
```

5.2 Cum se comportă BatchNorm la train vs eval

```
# BatchNorm are DOUA moduri de functionare, controlate de model.train() / model.eval():
#
# - In modul "train": foloseste media si varianta calculate pe BATCH-UL CURENT.
#   In paralel, actualizeaza niste statistici "running" (running_mean, running_var)
#   printr-o medie mobila exponentiala.
#
# - In modul "eval":  foloseste running_mean / running_var (statistici "inghetate"),
#   NU statisticile batch-ului curent. Acest lucru este esential pentru ca predictia
#   sa fie deterministica si sa nu depinda de ce alte exemple sunt in batch.
#
# Exact din acest motiv este OBLIGATORIU sa apelati model.eval() inainte de validare/inferenta -
# altfel performanta poate parea ciudat de slaba sau de instabila.
model.train()   # <- la inceputul fiecarei epoci de antrenare
model.eval()    # <- la inceputul evaluarii / inferentei
```

5.3 Interacțiunea cu dimensiunea batch-ului

```
# BatchNorm estimeaza media si varianta DIN BATCH. Daca batch-ul este foarte mic
# (ex: batch_size=2 sau batch_size=4), aceste estimari devin zgomotoase si BN
# poate INRAUTATI antrenarea in loc sa o ajute.
#
# Reguli empirice:
# - batch_size >= 32 -> BatchNorm functioneaza foarte bine
# - batch_size <  16 -> luati in considerare alternative: GroupNorm sau LayerNorm
```

```
#
# Exemplu de inlocuire BN -> GroupNorm cand suntem fortati la batch-uri mici
# (ex: imagini mari care nu incap in VRAM):
# nn.GroupNorm(num_groups=8, num_channels=64)   # 64 canale impartite in 8 grupuri
#
# ATENTIE: BatchNorm necesita batch_size > 1 ca sa poata calcula varianta.
# Daca ultimul batch are dimensiunea 1, antrenarea va da eroare.
# Solutie: drop_last=True in DataLoader.
train_loader = DataLoader(train_set, batch_size=64, shuffle=True,
                          num_workers=2, drop_last=True)
```

5.4 BatchNorm și fine-tuning

```
# Cand faceti fine-tuning pe un model pre-antrenat (ex: ResNet de la Sectiunea 5),
# straturile BatchNorm contin DEJA statistici invatate pe ImageNet (running_mean, running_var).
# Daca setul vostru de date este MIC, recalcularea acestor statistici cu batch-uri mici
# poate distruge informatia pre-antrenata.
#
# Solutia uzuala: "inghetam" straturile BatchNorm in modul eval pe tot parcursul antrenarii,
# chiar daca restul retelei este in modul train.
def freeze_bn(model):
    for module in model.modules():
        if isinstance(module, (nn.BatchNorm1d, nn.BatchNorm2d, nn.BatchNorm3d)):
            module.eval()                              # foloseste running stats, nu cele de batch
            for p in module.parameters():
                p.requires_grad = False                # opritea actualizarea lui gamma, beta

# Apel tipic in bucla de antrenare:
model.train()       # toata reteaua in modul train
freeze_bn(model)    # ... mai putin straturile BN, care raman "inghetate"
```

5.5 Observații practice

```
# Cateva sfaturi de retinut:
# - Ordinea uzuala: Conv -> BatchNorm -> ReLU. Varianta "BN dupa ReLU" mai exista,
#   dar in practica ordinea de mai sus este cea mai folosita si recomandata.
# - Nu puneti Dropout INTRE Conv si BN; dropout-ul interfereaza cu statisticile.
#   Daca folositi ambele, puneti Dropout-ul pe straturile fully-connected, dupa BN.
# - Pentru MLP-uri folositi nn.BatchNorm1d (input 2D), pentru CNN-uri nn.BatchNorm2d (input 4D).
```

```
# - Cand salvati / incarcati modelul cu state_dict(), running_mean si running_var sunt
#   salvate automat - nu trebuie sa faceti nimic special.
```

Resursă utilă: tutorial vizual pe d2l.ai

---

6. Finetuning pe MLP-uri și CNN-uri pre-antrenate

Atunci când setul de date este mic (cazul vostru), antrenarea de la zero este de obicei o idee proastă. În schimb, luăm un model pre-antrenat pe un set de date mare (de obicei **ImageNet**) și îl adaptăm pe sarcina noastră. Această tehnică se numește **transfer learning** sau **fine-tuning**.

6.1 Încărcarea unui model pre-antrenat și înlocuirea capului de clasificare

```
import torchvision.models as models
import torch.nn as nn

# Incarcam un ResNet18 pre-antrenat pe ImageNet
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# Modelul are pe ultimul strat 1000 clase (ImageNet). Trebuie sa il inlocuim
# cu un strat care are NUMARUL VOSTRU de clase (ex: 2 pentru clasificare binara).
num_classes = 2
in_features = model.fc.in_features          # numarul de features inainte de stratul final
model.fc = nn.Linear(in_features, num_classes)

model = model.to(device)
```

6.2 Adaptarea formei intrării

Modelele pre-antrenate pe ImageNet așteaptă intrări de forma **(3, 224, 224)** (3 canale RGB, normalizate cu statisticile ImageNet). Imaginile voastre din temă sunt mai mici și pot fi alb-negru (grayscale) — trebuie să le aducem la formatul așteptat de model.

```
# Cazul 1: imaginile voastre sunt RGB dar la rezolutie mica (ex: 64x64).
# Solutia simpla: facem resize la 224x224 cu transforms.Resize.
# (Pentru CNN-uri pur convolutionale ati putea evita resize-ul, dar ResNet are
#  un AdaptiveAvgPool care necesita o dimensiune minima, deci resize-ul e cel mai sigur.)
finetune_transform = transforms.Compose([
    transforms.Resize((224, 224)),                      # adaptam dimensiunea
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],    # OBLIGATORIU statistici ImageNet
```

```
                        std=[0.229, 0.224, 0.225]),    # daca modelul a fost antrenat asa
])
```

```
# Cazul 2: imaginile sunt GRAYSCALE (1 canal), modelul e RGB (3 canale).
# Avem doua optiuni curate:

# Optiunea A (recomandat pentru tema): replicam canalul gri de 3 ori.
# Asta pastreaza arhitectura ResNet intacta si functioneaza imediat.
gray_to_rgb_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=3),   # 1 canal -> 3 canale identice
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

# Optiunea B: modificam primul strat conv ca sa accepte 1 canal in loc de 3.
# Trebuie sa decidem ce facem cu ponderile pre-antrenate (3 canale).
# Cea mai folosita strategie: mediem ponderile pe dimensiunea de canal de intrare.
import torch
old_conv = model.conv1                                  # Conv2d(3, 64, kernel=7, stride=2, ...)
new_conv = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
with torch.no_grad():
    # mean pe axa canalelor de intrare (dim=1) pastreaza informatia "macroscopica"
    new_conv.weight[:] = old_conv.weight.mean(dim=1, keepdim=True)
model.conv1 = new_conv
```

6.3 Înghețarea backbone-ului vs fine-tuning complet

```
# Strategia 1: "feature extraction" - inghetam toata reteaua si antrenam doar capul.
# Util cand setul de date este foarte mic.
for param in model.parameters():
    param.requires_grad = False
# Doar parametrii noului strat fc raman cu requires_grad=True (sunt nou-creati)
for param in model.fc.parameters():
    param.requires_grad = True

# Optimizatorul vede DOAR parametrii antrenabili - filtram explicit:
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
```

```
# Strategia 2: fine-tuning cu learning rates diferite (recomandat).
# Stratul final (nou-initializat) are nevoie de un learning rate MAI MARE,
# iar backbone-ul (deja antrenat) de unul mai MIC, ca sa nu distrugem feature-urile bune.
backbone_params = [p for n, p in model.named_parameters() if not n.startswith("fc.")]
head_params     = [p for n, p in model.named_parameters() if     n.startswith("fc.")]

optimizer = optim.Adam([
    {"params": backbone_params, "lr": 1e-4},   # lr mic pe backbone
    {"params": head_params,     "lr": 1e-3},   # lr mai mare pe capul nou
], weight_decay=1e-4)
```

6.4 Warmup și scheduling pentru learning rate

La fine-tuning, gradienții inițiali mari de la capul nou-inițializat pot perturba ponderile pre-antrenate. Un **warmup** scurt (creșterea liniară a LR-ului în primele câteva epoci) urmat de un decay (de obicei cosine) ajută foarte mult.

```
from torch.optim.lr_scheduler import LambdaLR, CosineAnnealingLR, SequentialLR

warmup_epochs = 3
total_epochs  = 30

# Faza 1: warmup liniar de la 0 -> lr-ul setat in optimizer
warmup = LambdaLR(optimizer, lr_lambda=lambda e: (e + 1) / warmup_epochs)
# Faza 2: cosine decay pana la final
cosine = CosineAnnealingLR(optimizer, T_max=total_epochs - warmup_epochs)

# SequentialLR il leaga: primele 3 epoci warmup, apoi cosine
scheduler = SequentialLR(optimizer, schedulers=[warmup, cosine], milestones=[warmup_epochs])

for epoch in range(total_epochs):
    train_one_epoch(model, train_loader, criterion, optimizer, device)
    evaluate(model, val_loader, criterion, device)
    scheduler.step()   # apelam scheduler-ul O DATA PE EPOCA, dupa pas
```

6.5 Fine-tuning pe un MLP pre-antrenat

Aceeași idee se aplică și la MLP-uri: încărcăm ponderile, înlocuim capul, eventual înghețăm primele straturi.

```
# Presupunem ca avem un MLP pre-antrenat pe o sarcina similara,
# cu un fisier de checkpoint salvat anterior cu torch.save(model.state_dict(), ...).
pretrained_mlp = SimpleMLP(num_classes=10)            # arhitectura veche, 10 clase
```

```python
pretrained_mlp.load_state_dict(torch.load("pretrained_mlp.pt"))

# Inlocuim ultimul Linear (capul de clasificare) cu unul potrivit pentru tema noastra
old_head = pretrained_mlp.net[-1]                    # ultimul Linear din Sequential
pretrained_mlp.net[-1] = nn.Linear(old_head.in_features, num_classes)

# (optional) inghetam straturile timpurii
for layer in list(pretrained_mlp.net)[:-1]:
    for p in layer.parameters():
        p.requires_grad = False
```

Resursă utilă: PyTorch — Tutorial pentru Transfer Learning în Computer Vision

---

7. Introducere scurtă în curriculum learning

Curriculum learning este inspirat de modul în care învață oamenii: începem cu exemple **ușoare** și introducem treptat exemple mai **grele**. Empiric, această abordare conduce adesea la convergență mai rapidă și la o generalizare mai bună, mai ales pe seturi de date zgomotoase sau dezechilibrate.

Pentru a-l implementa aveți nevoie de (a) o noțiune de *dificultate* per exemplu și (b) o strategie care să expună modelul progresiv la exemple mai grele.

7.1 Definirea „dificultății”

```python
# "Dificultatea" depinde de problema. Cateva surse uzuale:
# - Marimea obiectului in imagine (pentru fete: fete mici = greu).
# - Variatia de iluminare / contrast (mole-uri foarte intunecate sau foarte slabe).
# - Loss-ul curent al modelului pe acel exemplu (auto-paced learning).
# - Scoruri furnizate de un model pre-antrenat (cu cat e mai mica increderea, cu atat e mai greu).

# Exemplu simplu: scor de dificultate dat manual la incarcarea dataset-ului.
class DatasetWithDifficulty(Dataset):
    def __init__(self, samples, difficulties, transform=None):
        # samples = lista (path, label), difficulties = lista de float-uri in [0, 1]
        assert len(samples) == len(difficulties)
        self.samples = samples
        self.difficulties = difficulties
        self.transform = transform

    def __len__(self):
        return len(self.samples)
```

```
    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert("RGB")
        if self.transform is not None:
            img = self.transform(img)
        return img, label, self.difficulties[idx]
```

7.2 Strategia 1 — funcție de pacing (filtrare în funcție de epocă)

```
from torch.utils.data import Subset

# Definim o "functie de pacing": ce procent din date (de la cele mai usoare la cele mai grele)
# este expus modelului la epoca curenta.
def pacing(epoch, total_epochs, start_frac=0.3):
    # Liniar de la start_frac la 1.0 - dupa jumatate de antrenare avem deja toate datele.
    return min(1.0, start_frac + (1 - start_frac) * (epoch / (total_epochs * 0.5)))

# Sortam indecsii dupa dificultate (cei usori intai)
sorted_indices = sorted(range(len(train_set)), key=lambda i: train_set.difficulties[i])

for epoch in range(total_epochs):
    frac = pacing(epoch, total_epochs)
    n_use = int(frac * len(sorted_indices))
    current_indices = sorted_indices[:n_use]            # primele n_use sample-uri (cele usoare)

    # Construim un sub-dataloader pentru epoca aceasta
    subset = Subset(train_set, current_indices)
    loader = DataLoader(subset, batch_size=64, shuffle=True, num_workers=2)

    train_one_epoch(model, loader, criterion, optimizer, device)
    evaluate(model, val_loader, criterion, device)
    # Observatie: validarea se face MEREU pe setul COMPLET, nu pe subsetul curent.
```

7.3 Strategia 2 — self-paced learning (folosim loss-ul propriu al modelului)

```
# Idee: la inceputul fiecarei epoci, calculam loss-ul per-exemplu cu modelul curent.
# Antrenam apoi doar pe exemplele cu loss sub un prag care creste in timp.
@torch.no_grad()
```

```python
def per_sample_losses(model, dataset, criterion_no_red, device, batch_size=128):
    model.eval()
    losses = []
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=False)
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        logits = model(images)
        # reduction="none" -> primim un loss per exemplu
        l = criterion_no_red(logits, labels)
        losses.append(l.cpu())
    return torch.cat(losses)

criterion_no_red = nn.CrossEntropyLoss(reduction="none")

for epoch in range(total_epochs):
    losses = per_sample_losses(model, train_set, criterion_no_red, device)
    # Pragul creste cu epoca -> incepem cu exemple usoare, apoi le includem si pe cele grele.
    threshold = torch.quantile(losses, q=min(1.0, 0.3 + 0.05 * epoch))
    easy_idx = (losses <= threshold).nonzero(as_tuple=True)[0].tolist()

    subset = Subset(train_set, easy_idx)
    loader = DataLoader(subset, batch_size=64, shuffle=True)
    train_one_epoch(model, loader, criterion, optimizer, device)
```

7.4 Observații practice

```python
# Cateva observatii importante:
# - Curriculum learning ajuta ADESEA, dar nu mereu. Faceti intotdeauna un baseline FARA curriculum.
# - Definirea "dificultatii" este partea grea; un proxy slab poate inrautati antrenarea.
# - Evaluarea se face MEREU pe setul de validare COMPLET, nu pe subsetul folosit la antrenare.
# - Curriculum-ul poate fi combinat cu augmentari (incepem cu augmentari blande, le facem mai
#   agresive ulterior - ceea ce se cheama "data augmentation curriculum").
```

Resursă utilă: Curriculum Learning in Neural Networks — definiții, principii, formulări matematice, considerente de implementare

---

Cum punem totul cap-coadă

Pentru temă, o progresie rezonabilă ar fi:

1. Începeți cu **Secțiunea 1** — puneți pe picioare un MLP simplu și un CNN simplu care se antrenează pe imaginile brute.
2. Adăugați **Secțiunea 2** (TensorBoard) ca să puteți compara experimentele clar.
3. Adăugați **Secțiunea 5** (BatchNorm) în arhitectura CNN-ului — ar trebui să observați convergență mai rapidă și posibilitatea de a folosi un learning rate mai mare.
4. Adăugați **Secțiunea 3** (augmentări) și veți observa cum se reduce diferența între curbele de train și val.
5. Dacă setul vostru de date este dezechilibrat (mai ales pentru sarcina cu mole-uri), aplicați **Secțiunea 4**.
6. Odată ce modelul antrenat de la zero atinge un platou, treceți la **Secțiunea 6** (fine-tuning pe ResNet/MobileNet) — de obicei aici apare cel mai mare salt de acuratețe pe seturile mici.
7. Încercați **Secțiunea 7** (curriculum) doar peste un baseline care funcționează deja, ca rafinament suplimentar.